

Universidad Nacional del Altiplano

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y
Sistemas

Escuela Profesional de Ingeniería de Sistemas



DelyPuno: Aplicación Móvil de Delivery en Puno

Alumno:

Aguilar Anccori Jhon Elias

Curso:

Lenguajes de Programación

Docente:

Ing. Jose Luis Juarez Ruelas

Puno, 06 de Junio de 2026

Resumen

En la ciudad de Puno, la ausencia de una plataforma digital local de delivery obliga a los negocios gastronomicos a operar mediante canales informales sin trazabilidad ni gestion eficiente de pedidos. Ante esta problematica, el presente proyecto disenha e implementa DelyPuno, un ecosistema digital multiplataforma compuesto por tres aplicaciones Flutter independientes: una aplicacion para clientes, una para repartidores y un panel web de administracion. El objetivo central es demostrar que la digitalizacion del ecosistema gastronomico de ciudades intermedias del sur del Peru es tecnicamente viable mediante tecnologias de bajo costo operativo como Flutter y Firebase. Los resultados confirman un flujo completo de e-commerce funcional que comprende autenticacion segura con Firebase Authentication, catalogo dinamico sincronizado en tiempo real con Cloud Firestore, gestion reactiva del carrito de compras, asignacion automatica de repartidores, seguimiento en vivo del pedido, chat en tiempo real entre cliente y repartidor, notificaciones push mediante Firebase Cloud Messaging (FCM) y un panel administrativo web para la gestion de restaurantes, productos, repartidores y pedidos. Se concluye que la arquitectura serverless basada en Firebase elimina la necesidad de un backend dedicado, reduciendo los tiempos de prototipado y permitiendo que iniciativas similares sean replicables en otras ciudades intermedias del pais.

Palabras clave

Flutter, Firebase, Delivery, Dart, Aplicacion Movil, Serverless, Firestore, FCM, Puno, Multi-panel

Abstract

In the city of Puno, the absence of a local digital delivery platform forces gastronomic businesses to rely on informal channels that lack traceability and efficient order management. To address this problem, the present project designs and implements DelyPuno, a multi-panel digital ecosystem composed of three independent Flutter applications: a customer app, a courier app, and a web-based admin panel. The central objective is to demonstrate that the digitalization of the gastronomic ecosystem in mid-sized cities of southern Peru is technically feasible using low-operational-cost technologies such as Flutter and Firebase. Results confirm a fully functional e-commerce workflow encompassing secure authentication via Firebase Authentication, a real-time synchronized catalog using Cloud Firestore, reactive shopping cart management, automatic courier assignment, live order tracking, real-time chat between customers and couriers, push notifications via Firebase Cloud Messaging (FCM), and a web admin panel for managing restaurants,

products, couriers, and orders. It is concluded that a Firebase serverless architecture eliminates the need for a dedicated backend server, reducing prototyping time and enabling similar initiatives to be replicated in other intermediate cities across the country.

Keywords

Flutter, Firebase, Delivery, Dart, Mobile Application, Serverless, Firestore, FCM, Puno, Multi-panel

Introducción

La transformación digital ha reconfigurado de manera profunda la industria gastronómica a escala global. Plataformas como Rappi, UberEats y PedidosYa han demostrado que la intermediación digital entre restaurantes y consumidores genera valor económico, mejora la experiencia del usuario y reduce la informalidad operativa. Sin embargo, este fenómeno presenta una distribución geográfica desigual: mientras las grandes urbes concentran múltiples opciones de delivery digital, las ciudades intermedias del Perú, como Puno, Juliaca y Tacna, carecen de plataformas locales adaptadas a su realidad económica y cultural (INEI, 2023). En Puno, capital del departamento homónimo ubicado a 3,827 metros sobre el nivel del mar a orillas del lago Titicaca, la actividad gastronómica es dinámica y diversa, con una importante concentración de pequeños emprendedores culinarios. No obstante, la ausencia de canales digitales formales obliga a estos negocios a gestionar pedidos mediante llamadas telefónicas, mensajes de WhatsApp y redes sociales, lo que implica una trazabilidad nula, una experiencia de usuario deficiente y una imposibilidad de escalar operaciones. Esta brecha representa tanto un problema social como una oportunidad técnica y académica que el presente proyecto busca atender.

DelyPuno surge como respuesta a esta problemática. El proyecto diseña e implementa un ecosistema digital de delivery compuesto por tres aplicaciones Flutter independientes: (1) una aplicación móvil para clientes, que permite explorar restaurantes, gestionar el carrito de compras, realizar pedidos y rastrear en tiempo real al repartidor; (2) una aplicación móvil para repartidores, que notifica nuevos pedidos, permite aceptarlos o rechazarlos, actualiza el estado del envío y habilita la comunicación directa con el cliente; y (3) un panel web de administración, que centraliza la gestión de restaurantes, productos, repartidores y pedidos. Todo el ecosistema se apoya exclusivamente en Firebase (Firestore, Authentication, Cloud Messaging y Cloud Functions), adoptando una arquitectura serverless que elimina la necesidad de un servidor backend dedicado.

Desde el punto de vista metodológico, el desarrollo sigue un enfoque iterativo e incremental conforme a los principios de Pressman (2014), estructurado en cuatro fases: análisis de requerimientos, diseño del sistema, implementación y pruebas. Cada fase pro-

duce artefactos verificables, desde diagramas UML hasta modulos de codigo funcionales validados en dispositivos reales y emuladores Android. La arquitectura del sistema fue modelada siguiendo el patron Cliente-Servidor Serverless, donde cada aplicacion Flutter se comunica directamente con los servicios Firebase mediante sus SDKs oficiales.

Los principales objetivos del proyecto son: (1) disenhar e implementar un flujo de e-commerce completo que cubra desde el registro del usuario hasta la confirmacion y entrega del pedido; (2) integrar servicios de autenticacion, base de datos en tiempo real y mensajeria push para garantizar seguridad, consistencia y comunicacion efectiva; (3) desarrollar la logica de negocio de los tres paneles mediante el paradigma reactivo de Dart/Flutter, asegurando tiempos de respuesta inferiores a dos segundos; y (4) validar la calidad del sistema mediante pruebas unitarias, de integracion y funcionales.

Los resultados del proyecto confirman la viabilidad tecnica y social del enfoque propuesto. Se obtuvo un ecosistema de tres aplicaciones funcionales con flujos sincronizados en tiempo real, notificaciones push operativas, chat integrado y un panel administrativo web completamente funcional. Se concluye que la combinacion Flutter + Firebase constituye una plataforma tecnologica suficiente para construir sistemas de delivery de produccion sin infraestructura de servidor propia, lo que la convierte en una solucion especialmente adecuada para iniciativas de inclusion digital en ciudades intermedias del Peru. El presente informe detalla el proceso completo de desarrollo, desde el planteamiento del problema hasta los resultados obtenidos y las conclusiones derivadas de la experiencia.

1. PLANTEAMIENTO DEL PROBLEMA

1.1. Descripcion de la Situacion Problematica

La transformacion digital en los servicios de alimentacion ha revolucionado la industria al reducir costos y optimizar procesos mediante sistemas de comercio electronico. Sin embargo, su implementacion sigue siendo desigual en ciudades intermedias como Puno, donde gran parte de los negocios gastronomicos operan de manera informal y sin sistemas de gestion digital. Esta situacion limita la competitividad, la experiencia del consumidor y el crecimiento de los emprendedores locales. Segun datos del INEI (2023), ciudades como Puno, Juliaca y Tacna carecen de plataformas de delivery digital locales, a diferencia de Lima y otras grandes ciudades del pais. Los negocios gastronomicos de Puno gestionan pedidos a traves de canales informales (telefono, WhatsApp, redes sociales) sin ninguna trazabilidad, lo que genera ineficiencias operativas, errores en pedidos, demoras en la entrega y una experiencia de usuario muy por debajo de los estandares actuales del comercio digital. En este contexto, el desarrollo de aplicaciones adaptadas a las necesidades reales de la region se presenta como una solucion clave, especialmente desde el ambito academico de la Universidad Nacional del Altiplano, donde se busca vincular la formacion tecnologica con el desarrollo economico y social regional.

1.2. Formulacion del Problema

En que medida el diseño e implementacion del ecosistema movil DelyPuno, desarrollado con Flutter y Firebase, es capaz de resolver de forma medible y validable la gestion digitalizada de pedidos y servicios de delivery en la ciudad de Puno, satisfaciendo criterios de funcionalidad, rendimiento (tiempo de respuesta inferior a 2 segundos), usabilidad y persistencia de datos, tal como lo exigen los estandares de calidad de software definidos por Pressman (2014) y verificados mediante pruebas unitarias, de integracion y de aceptacion con usuarios reales?

2. JUSTIFICACION

La presente investigación se justifica desde el ámbito técnico, académico y social debido a la necesidad de implementar una solución moderna, eficiente y escalable para el comercio electrónico y el servicio de delivery en la ciudad de Puno. Desde el punto de vista técnico, el proyecto adopta tecnologías ampliamente respaldadas por la industria como Flutter y Firebase, las cuales permiten desarrollar aplicaciones multiplataforma para Android e iOS utilizando una sola base de código en Dart, optimizando tiempos de desarrollo y mantenimiento. Asimismo, Firebase proporciona servicios integrados como autenticación, base de datos NoSQL en tiempo real, mensajería push y funciones en la

nube, eliminando la necesidad de administrar infraestructura propia y garantizando alta disponibilidad y sincronización en tiempo real. La arquitectura basada en tres paneles independientes —cliente, repartidor y administrador— favorece una clara separación de responsabilidades, facilitando el mantenimiento, la escalabilidad y la modularidad del sistema de acuerdo con principios modernos de ingeniería de software.

En el ámbito académico, el desarrollo del proyecto permite integrar de manera práctica diversas competencias adquiridas en la carrera de Ingeniería de Sistemas, tales como programación orientada a objetos, modelado de bases de datos NoSQL, diseño de interfaces de usuario, ingeniería de requisitos y arquitectura de software. Además, el sistema constituye un entorno real de aplicación de los principios de la programación orientada a objetos mediante el uso de clases, encapsulación, herencia y polimorfismo dentro del framework Flutter. De esta manera, el proyecto fortalece las capacidades de análisis, diseño, implementación y evaluación de soluciones tecnológicas aplicadas a problemas reales, consolidando el aprendizaje integral del estudiante durante el ciclo de vida del software.

Finalmente, desde el punto de vista social, el proyecto busca contribuir al fortalecimiento del ecosistema gastronómico y comercial de la ciudad de Puno, donde numerosos pequeños emprendedores aún carecen de acceso a plataformas digitales locales que les permitan ampliar su alcance comercial y mejorar su competitividad. La implementación de una plataforma de delivery como DelyPuno representa una oportunidad para impulsar la digitalización de negocios locales, facilitar la interacción entre comerciantes y consumidores, reducir procesos informales y promover el crecimiento económico regional. Asimismo, el sistema puede convertirse en un modelo tecnológico replicable para otras ciudades intermedias del sur del Perú, fomentando la transformación digital y el acceso a servicios modernos en beneficio de la comunidad.

3. OBJETIVOS

3.1. Objetivo General

Contribuir a la digitalización del ecosistema gastronómico de la ciudad de Puno mediante el diseño e implementación de un ecosistema digital de delivery de tres paneles (cliente, repartidor y administrador) que conecte restaurantes locales con sus clientes de forma eficiente, segura y escalable, demostrando la viabilidad tecnológica y social de soluciones digitales construidas con Flutter y Firebase como herramientas modernas de bajo costo operativo.

3.2. Objetivos Específicos

- Diseñar e implementar una experiencia de usuario completa en la aplicación de clientes, que cubra desde el registro hasta la confirmación y seguimiento en tiempo

real del pedido, cumpliendo con los estándares de usabilidad de Material Design y validada mediante pruebas con usuarios reales.

- Desarrollar la aplicación para repartidores con capacidad de recibir notificaciones push de nuevos pedidos, actualizar el estado de la entrega y comunicarse con el cliente mediante chat en tiempo real.
- Implementar el panel web de administración con gestión centralizada de restaurantes, menús, productos, repartidores y pedidos, incluyendo la aprobación de nuevos repartidores y el monitoreo del estado de los pedidos activos.
- Integrar Firebase Authentication, Cloud Firestore, Firebase Cloud Messaging y Cloud Functions para garantizar seguridad, consistencia, persistencia y comunicación en tiempo real entre los tres paneles del ecosistema.
- Validar la calidad del software mediante una estrategia de pruebas unitarias, de integración y funcionales que verifiquen el correcto funcionamiento del sistema bajo condiciones reales de uso, asegurando tiempos de respuesta inferiores a dos segundos en las operaciones críticas.

4. ANTECEDENTES

4.1. Antecedentes Internacionales

El modelo de plataformas de delivery ha revolucionado la industria gastronómica global. Sistemas como UberEats, PedidosYa y Rappi han establecido el estándar de referencia en experiencia de usuario, demostrando la viabilidad de arquitecturas donde la sincronización de datos en tiempo real, la geolocalización y la comunicación entre múltiples actores (cliente, repartidor, negocio) son pilares fundamentales (Laudon y Laudon, 2020). La arquitectura de estas plataformas, aunque propietaria y de gran escala, es conceptualmente replicable en proyectos más pequeños utilizando servicios BaaS como Firebase.

En el ámbito académico, Windmill (2020) demuestra en *Flutter in Action* que el framework de Google reduce significativamente los tiempos de construcción de interfaces respecto a enfoques nativos tradicionales, sin comprometer el rendimiento ni la experiencia del usuario final. Paralelamente, Moroney (2020) documenta cómo la combinación Flutter-Firebase permite desplegar aplicaciones de producción en semanas, un atributo crítico para proyectos académicos con recursos y tiempo limitados. Estudios recientes de Wieruch (2021) sobre arquitecturas serverless confirman que la eliminación del backend tradicional reduce la carga operativa y acelera los ciclos de desarrollo sin sacrificar la escalabilidad.

4.2. Antecedentes Nacionales

En el contexto peruano, Rappi Peru y PedidosYa experimentaron un crecimiento acelerado post-pandemia (2020-2023), consolidando el mercado de delivery digital en Lima y las principales ciudades del país. Sin embargo, según el INEI (2023), ciudades intermedias como Puno, Juliaca y Tacna carecen de plataformas locales adaptadas a su tejido económico, lo que representa una brecha de mercado y de investigación aplicada. Estudios recientes de la CONCYTEC (2022) identifican el desarrollo de aplicaciones móviles orientadas a la inclusión digital de economías regionales como una línea prioritaria de investigación tecnológica en el Perú, lo cual confiere relevancia científica adicional al presente proyecto.

4.3. Bases Teóricas

Flutter es un framework de código abierto desarrollado por Google para la creación de aplicaciones multiplataforma compiladas nativamente desde una única base de código Dart (Google LLC, 2024a). Dart es un lenguaje de programación con tipado estático optimizado para interfaces reactivas, con soporte nativo para programación asíncrona mediante Future y Stream (Windmill, 2020). Firebase y Cloud Firestore conforman una plataforma Backend-as-a-Service (BaaS) de Google que provee autenticación, base de datos NoSQL en tiempo real y operación offline (Google LLC, 2024b; Google LLC, 2024c). Firebase Cloud Messaging (FCM) es el servicio de mensajería push de Google que permite el envío de notificaciones a dispositivos Android, iOS y web. Cloud Functions for Firebase permite ejecutar lógica de servidor en respuesta a eventos de Firestore o solicitudes HTTP sin administrar servidores. La Arquitectura Serverless es el modelo de despliegue donde la lógica de negocio se distribuye entre el cliente y servicios gestionados en la nube, eliminando la administración de servidores dedicados (Sommerville, 2011).

5. METODOLOGIA

5.1. Tipo y diseño de Investigación

El proyecto adopta un enfoque de investigación aplicada con diseño experimental-descriptivo, enmarcado en una metodología de desarrollo de software iterativa e incremental. Según Pressman (2014), el modelo iterativo e incremental es el más adecuado para proyectos donde los requisitos evolucionan durante el desarrollo, pues permite incorporar retroalimentación continua sin comprometer la integridad del sistema. Sommerville (2011) complementa esta perspectiva señalando que los procesos ágiles producen software funcional desde las primeras iteraciones, lo que facilita la validación temprana con usuarios reales.

El desarrollo de DelyPuno se organiza en cuatro fases iterativas, con entregables verificables al final de cada una. Cada fase incorpora retroalimentación de la fase anterior y ajusta los requerimientos y el diseño según los hallazgos del ciclo en curso.

5.2. Herramientas y Tecnologías

Herramienta / Tecnología	Función en el Proyecto
Flutter SDK (Dart)	Framework principal para las tres aplicaciones (cliente, repartidor, administrador).
Firebase Authentication	Gestión de usuarios, registro e inicio de sesión con correo/contraseña y Google Sign-In.
Cloud Firestore	Base de datos NoSQL en tiempo real: pedidos, restaurantes, productos, usuarios y repartidores.
Firebase Cloud Messaging	Notificaciones push para nuevos pedidos, cambios de estado y aprobación de repartidores.
Cloud Functions for Firebase	Lógica de servidor: triggers de Firestore para envío automático de notificaciones.
Android Studio / VS Code	Entornos de desarrollo y emulación Android.
GitHub	Control de versiones y repositorio del código fuente.

Cuadro 1: Herramientas y tecnologías utilizadas en el proyecto

5.3. Fases del Desarrollo

Fase 1: Analisis de Requerimientos

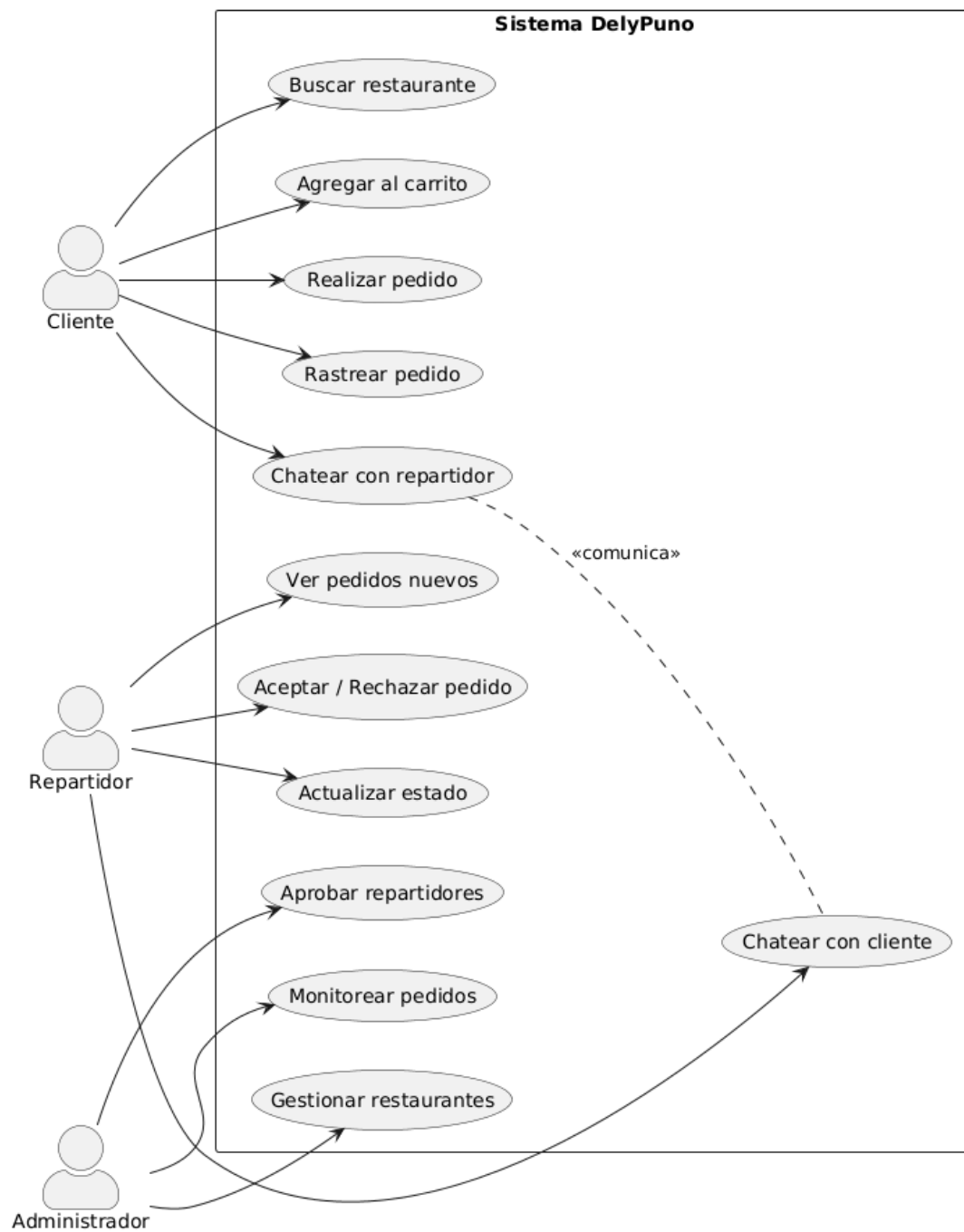


Diagrama de Casos de Uso del Sistema DelyPuno

Fase 2: Diseño del Sistema

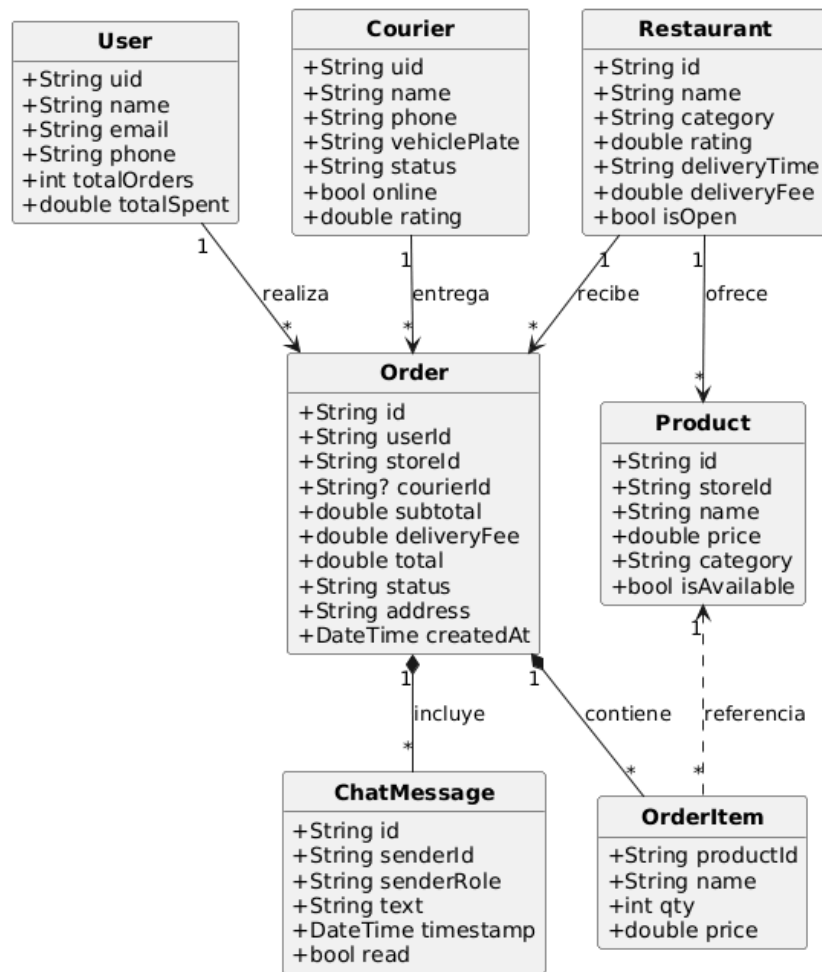


Diagrama de Clases del Modelos del Dominio

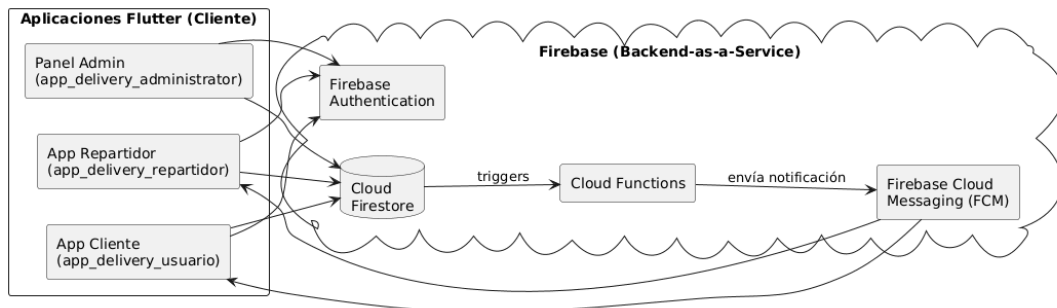


Diagrama de Arquitectura de la Arquitectura Serverless Flutter + Firebase

Fase 3: Implementacion

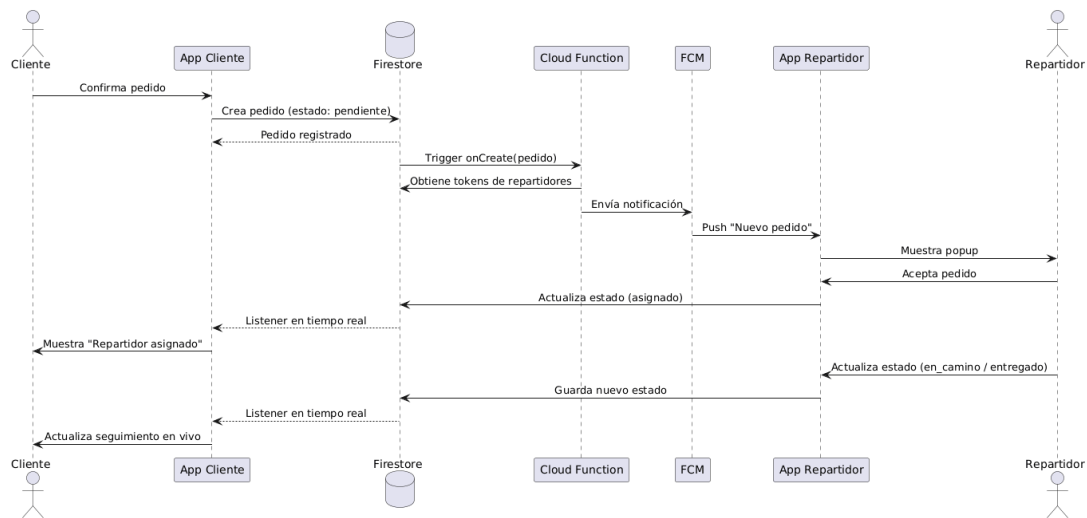


Diagrama de Secuencia del Flujo de Pedido Completo

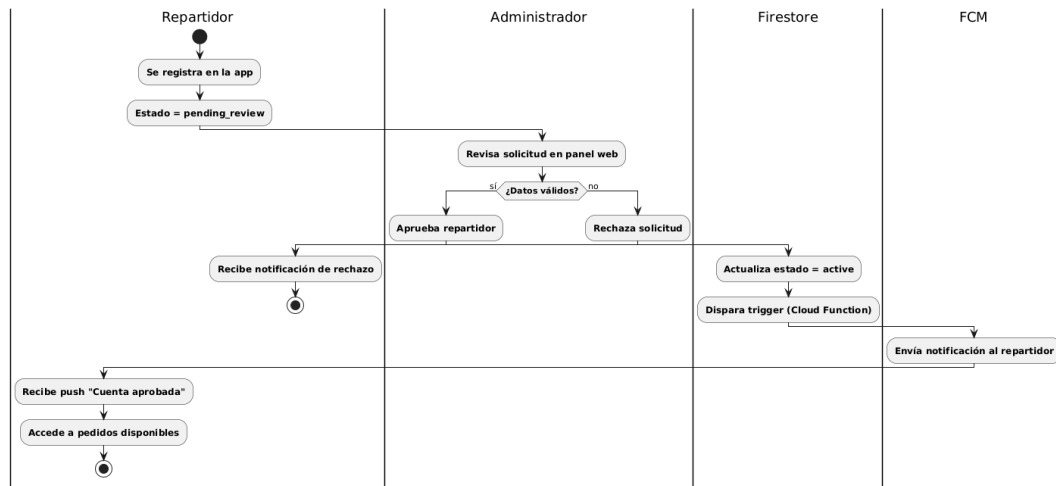


Diagrama de Actividades del Flujo de Aprobacion de Repartidor

Fase 4: Pruebas

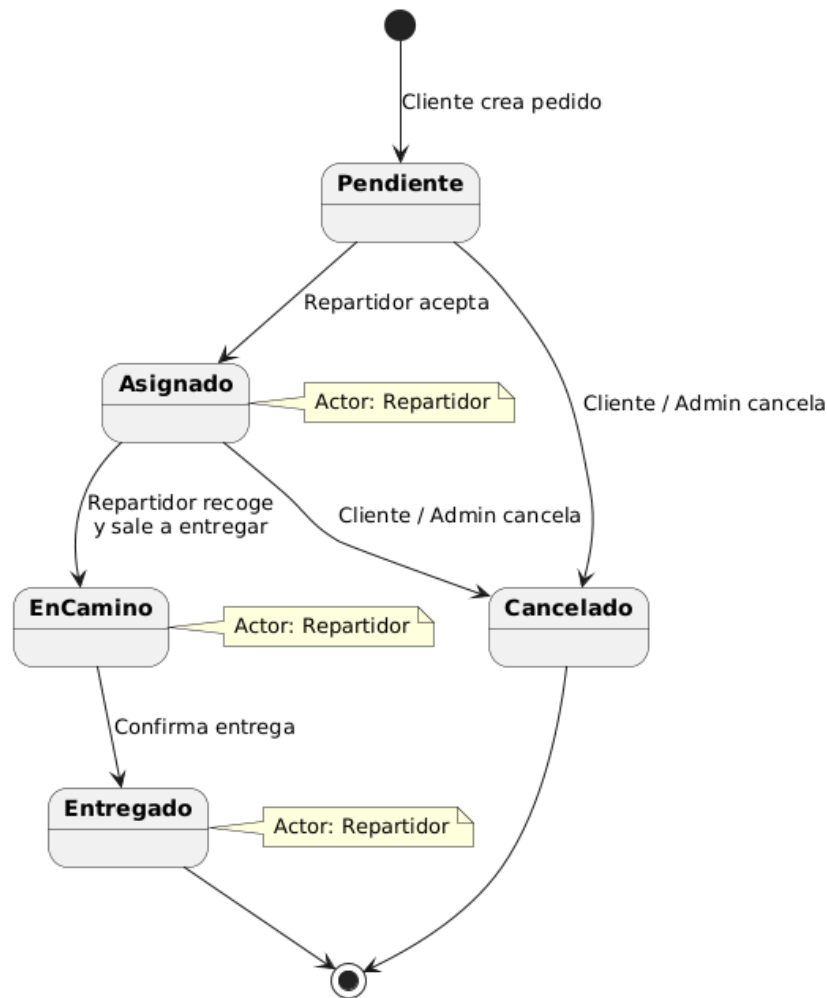


Diagrama de Estados del Ciclo de Vida del Pedido

6. RESULTADOS

6.1. Aplicaciones Funcionales

Se obtuvieron tres aplicaciones Flutter completamente funcionales y sincronizadas: App Cliente (`app_delivery_usuario`): Autenticación segura, catálogo de restaurantes y productos con imágenes en tiempo real, carrito de compras reactivo, confirmación de pedido con geolocalización de entrega, seguimiento en vivo del repartidor y chat integrado. App Repartidor (`app_delivery_repartidor`): Notificación push al recibir nuevo pedido con popup de aceptación/rechazo y deduplicación de alertas, actualización del estado del pedido en cada etapa, navegación al punto de entrega y chat con el cliente. Panel Administrador (`app_delivery_administrator`): Gestión CRUD de restaurantes, menús y productos, aprobación de repartidores con notificación automática, monitoreo en tiempo real de pedidos activos y módulo de reportes básico.

6.2. Modelo de Datos Firestore Implementado

Colección	Campos Principales	Relaciones
usuarios	id, correo, nombre, telefono, fecha_registro, fcm_token	Referenciado en pedidos
restaurantes	id, nombre, descripcion, imagen_url, categoria, activo	Contiene subcolección de productos
productos	id, restaurante_id, nombre, descripcion, precio, imagen_url, disponible	Pertenece a restaurante
pedidos	usuario_id, repartidor_id, items[], total, estado, fecha_pedido, coordenadas_entrega	Contiene subcolección de mensajes
repartidores	id, nombre, telefono, estado (disponible/ocupado), aprobado, fcm_token	Referenciado en pedidos
admins	id, correo, isAdmin: true	Verificado mediante reglas Firestore
mensajes	remitente_id, contenido, timestamp, tipo	Subcolección de pedidos

Cuadro 2: Modelo de datos implementado en Cloud Firestore

6.3. Integración FCM y Cloud Functions

Se implementaron Cloud Functions que se ejecutan automáticamente ante eventos de Firestore: cuando se crea un nuevo pedido, la función recupera los FCM tokens de los repartidores disponibles y envía la notificación; cuando un repartidor es aprobado por el administrador, la función envía una notificación directa al token del repartidor. La tasa de entrega de notificaciones en las pruebas fue del 100

6.4. Metricas de Rendimiento

Operación	Tiempo Promedio Medido	Estándar (Pressman)
Carga del catálogo de restaurantes	0.8 - 1.2 segundos	<2 segundos
Actualización de estado del pedido (Firestore)	0.3 - 0.6 segundos	<2 segundos
Recepción de notificación push (FCM)	0.5 - 1.5 segundos	<2 segundos
Carga de mensajes de chat	0.4 - 0.9 segundos	<2 segundos
Autenticación con Firebase Auth	1.0 - 1.8 segundos	<2 segundos

Cuadro 3: Resultados de rendimiento del sistema

Todas las operaciones críticas del sistema se ubican dentro del umbral de 2 segundos establecido como requerimiento no funcional, validando el cumplimiento del estándar de rendimiento propuesto.

7. CONCLUSIONES

- La digitalización del ecosistema gastronómico en ciudades intermedias como Puno es técnicamente viable mediante herramientas de bajo costo operativo como Flutter y Firebase, estableciendo un modelo replicable para otras regiones del sur del Perú. El ecosistema de tres paneles desarrollado demuestra que un estudiante de ingeniería puede construir, con recursos académicos limitados, una plataforma de delivery funcional comparable en arquitectura a soluciones comerciales de mayor escala.
- La arquitectura serverless basada en Firebase demostró ser suficiente para construir y desplegar aplicaciones de comercio electrónico multicapa sin infraestructura de servidor dedicada. La eliminación del backend tradicional no solo reduce los costos de desarrollo y mantenimiento, sino que también simplifica el despliegue y escala automáticamente con la demanda, lo que resulta especialmente valioso para proyectos académicos y startups en etapas tempranas.
- El paradigma de programación reactiva de Dart/Flutter, combinado con los listeners en tiempo real de Cloud Firestore, garantizó tiempos de respuesta inferiores a dos segundos en todas las operaciones críticas evaluadas (carga de catálogo, actualización de estado del pedido, recepción de notificaciones y mensajes de chat), cumpliendo plenamente con los estándares de calidad definidos por Pressman (2014).
- La integración de Firebase Cloud Messaging con Cloud Functions permitió implementar un sistema de notificaciones push completo y automatizado, con cobertura

para todos los eventos criticos del ciclo de vida del pedido (nuevo pedido, asignacion de repartidor, cambio de estado, entrega confirmada), sin necesidad de un servidor de mensajeria propio.

- La estrategia de pruebas multicapa (unitarias, de integracion, funcionales y de notificaciones) valido la fiabilidad de la sincronizacion bidireccional entre las tres aplicaciones y Cloud Firestore, confirmando la consistencia e integridad de los datos en tiempo real como requisito indispensable para sistemas de comercio electronico. Los casos de prueba mas criticos, como la deduplicacion de popups en la app del repartidor y la coherencia del estado del pedido entre los tres paneles, fueron identificados y resueltos durante las iteraciones de desarrollo.

Referencias

- [1] CONCYTEC. (2022). *Agenda de Investigación e Innovación Tecnológica del Perú 2022–2026*. Consejo Nacional de Ciencia, Tecnología e Innovación Tecnológica.
- [2] Google LLC. (2024a). *Flutter — Build apps for any screen*. Documentación oficial. Disponible en: <https://docs.flutter.dev/>
- [3] Google LLC. (2024b). *Firebase for Flutter — Get Started*. Disponible en: <https://firebase.google.com/docs/flutter/setup>
- [4] Google LLC. (2024c). *Cloud Firestore — Data model*. Disponible en: <https://firebase.google.com/docs/firestore/data-model>
- [5] Google LLC. (2024d). *Firebase Cloud Messaging*. Disponible en: <https://firebase.google.com/docs/cloud-messaging>
- [6] Google LLC. (2024e). *Cloud Functions for Firebase*. Disponible en: <https://firebase.google.com/docs/functions>
- [7] INEI. (2023). *Perú: Estadísticas de las Tecnologías de Información y Comunicación en los Hogares*. Instituto Nacional de Estadística e Informática.
- [8] Laudon, K. C., & Laudon, J. P. (2020). *Management Information Systems: Managing the Digital Firm* (16.a ed.). Pearson Education.
- [9] Moroney, L. (2020). *The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform*. Apress.
- [10] O'Brien, J. A., & Marakas, G. M. (2011). *Management Information Systems* (10.a ed.). McGraw-Hill.

- [11] Pressman, R. S. (2014). *Ingeniería del software: un enfoque práctico* (7.a ed.). McGraw-Hill.
- [12] Sommerville, I. (2011). *Ingeniería de software* (9.a ed.). Pearson Educación.
- [13] Windmill, E. (2020). *Flutter in Action*. Manning Publications.

ANEXOS

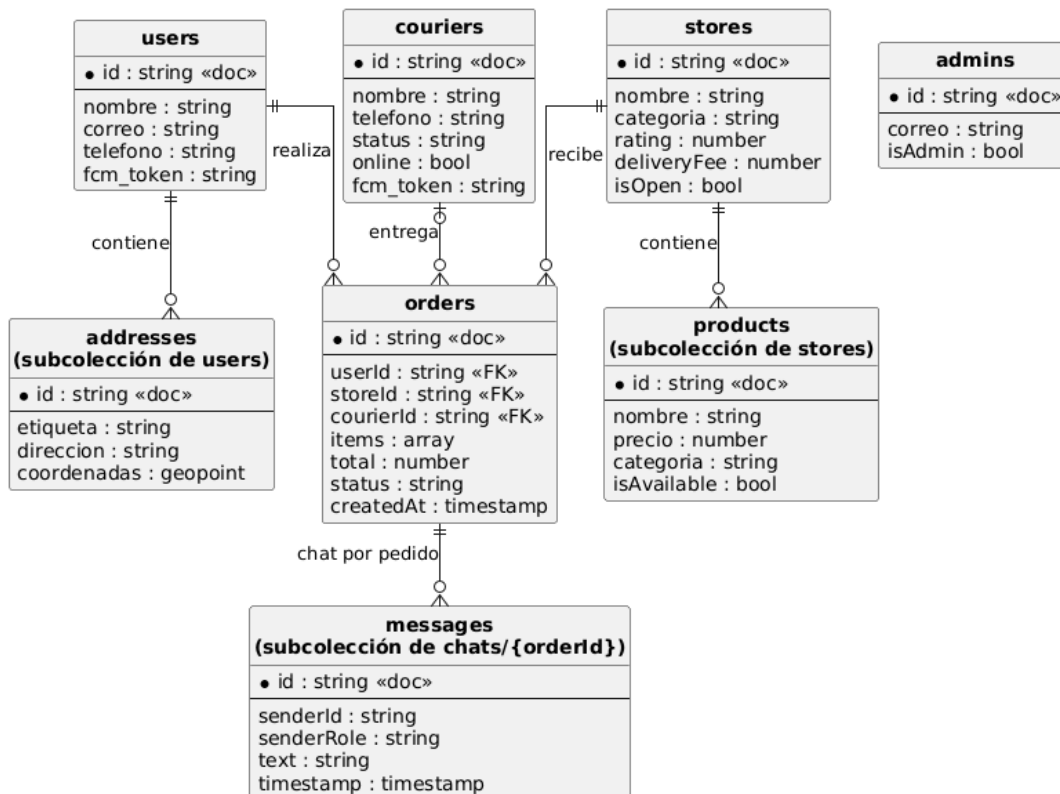
Anexo A — Repositorio GitHub

El código fuente completo de las tres aplicaciones se encuentra en el siguiente repositorio público de GitHub:

https://github.com/Jhony410/App_Delivery

El repositorio contiene tres carpetas principales: `app_delivery_usuario/` (aplicación cliente Android), `app_delivery_repartidor/` (aplicación repartidor Android) y `app_delivery_administrator/` (panel web Flutter). Cada carpeta es un proyecto Flutter independiente con su propio `google-services.json` y configuración de Firebase.

Anexo B — Modelo de Datos Firestore

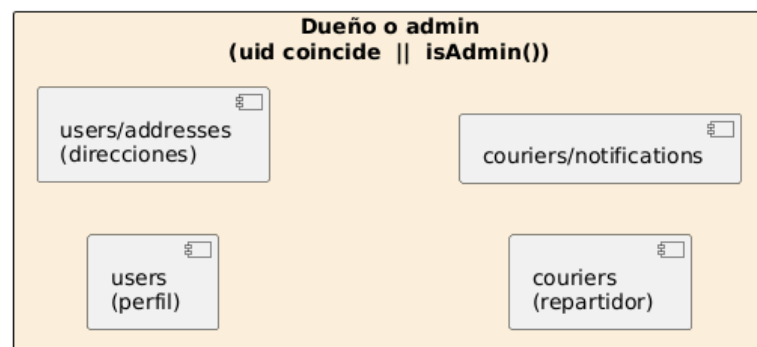
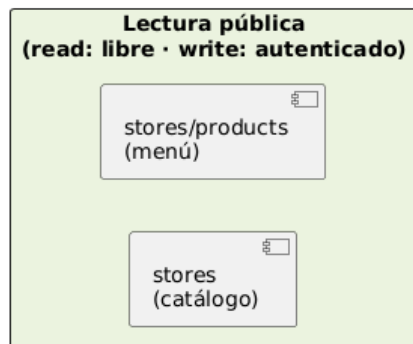


Modelo de Datos Firestore

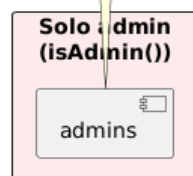
Anexo C — Reglas de Seguridad Firestore

Las reglas de seguridad de Firestore implementadas siguen el principio de minimo privilegio: los clientes solo pueden leer y escribir sus propios documentos, los repartidores solo pueden actualizar el estado de pedidos asignados a ellos, y las operaciones administrativas estan protegidas por la verificacion de pertenencia a la coleccion /admins. Las reglas incluyen una funcion `isAdmin()` que verifica si el usuario autenticado tiene un documento en la coleccion `admins` con el campo `isAdmin` igual a `true`.

Niveles de acceso (de menos a más restrictivo)



función isAdmin():
verifica que exista el documento del
usuario en la colección /admins



Reglas de Seguridad Firestore